

Dynamic Programming

情况一: x_i 和 y_j 配对[矩阵链乘-动态规划.pdf](#)

矩阵连乘问题:

问题背景:

给定 n 个矩阵构成的序列: $\langle A_1, A_2, \dots, A_n \rangle$, 计算它们的乘积:

$$A_1 A_2 \cdots A_n$$

由于矩阵乘法满足**结合律**, 所以不同的括号化方式不会改变最终结果。例如:

$$(A_1 A_2) A_3 = A_1 (A_2 A_3)$$

也就是说, 矩阵链乘问题中, **矩阵相乘的顺序可以不同, 但最终乘积结果相同**。对于多个矩阵相乘, 可以有多种不同的括号化方式。

■ 不同的括号化方案产生不同的计算成本(标量乘法次数)

两矩阵相乘 $A_{pq} \cdot B_{qr}$ 的**标量乘法次数**为 $p \times q \times r$

例: 设 A_1, A_2, A_3 的**维数**分别为 $10 \times 100, 100 \times 5, 5 \times 50$

$$(A_1 (A_2 A_3)): \quad A_2 A_3 \text{ — } 100 \times 5 \times 50 = 25000 \quad (\text{结果维数: } 100 \times 50)$$

$$A_1 (A_2 A_3) \text{ — } 10 \times 100 \times 50 = 50000$$

$$\text{Total: } 50000 + 25000 = 75000$$

$$((A_1 A_2) A_3): \quad A_1 A_2 \text{ — } 10 \times 100 \times 5 = 5000 \quad (\text{结果维数: } 10 \times 5)$$

$$(A_1 A_2) A_3 \text{ — } 10 \times 5 \times 50 = 2500$$

$$\text{Total: } 5000 + 2500 = 7500$$

■ 原问题等价于求最优(计算成本最小)的括号化方案

若两个矩阵分别为: $A_{p \times q}, B_{q \times r}$, 则它们可以相乘, 结果矩阵维度为: $p \times r$ 。所需的标量乘法次数为: $p \times q \times r$ 。因此, 矩阵乘法的计算成本主要由**三个维度决定**:

$$\text{行数} \times \text{中间维度} \times \text{列数}$$

所以我们的目的就是，怎么样分配括号可以让总的乘法数量最少，这就进入我们的**矩阵连乘问题**。

动态规划的基本求解步骤：

Step 1: 定义子问题空间

先明确：原问题可以拆成哪些**子问题**？在矩阵链乘中，后面会把： $A_i A_{i+1} \cdots A_j$ ，也就是其中连续的一段看作一个子问题。

Step 2: 刻画最优解的结构特征

这一步也叫寻找**最优子结构**。原问题的最优解，是否包含子问题的最优解？如果**大问题的最优解可以由小问题的最优解组合出来**，那么就适合动态规划。

❖ 假设 $A_i, A_{i+1}, \dots, A_j$ 的**最优括号化方案**在 A_k 和 A_{k+1} 之间进行分裂，则最优解为：

$$\underbrace{A_i, A_{i+1}, \dots, A_k}_{\text{最优括号化方案}} \times \underbrace{A_{k+1}, A_{i+1}, \dots, A_j}_{\text{最优括号化方案}}$$

只要分裂点 k 固定，最后一次相乘的成本 z 就固定。左边内部怎么加括号，右边内部怎么加括号，都不会改变最后两个结果矩阵的维度。

假设此时这样的分割策略就是最优的方案，如果左侧不是最优，那么左侧一定能够在优化，这样的优化会使得目前这一段变得更好，这就**与我们“已经最优”的假设矛盾了**，右侧也可以同样的看。

所以**每个最优的大问题里，更小规模的问题也是最优的**，所以这是一个合理的结构。

Step 3: 根据最优子结构写递归式

找到最优子结构后，就要写出**状态转移关系**。**大问题的答案如何由小问题的答案推出**？在矩阵链乘中，后面会得到递推式：

$$m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\}$$

特殊地， $i = j$ 时， $m[i, j] = 0$ 。因为链上只有一个矩阵，无需乘法。 k 代表的是最优分割点，对于 i, j 规模，他的最优值就是左半部分 + 右半部分 + 两部分的乘法消耗。（有没有梦回分治 *merge*）

Step 4: 自底向上计算最优解的值

有了递归式之后，不直接**递归暴算**，而是**从最小子问题开始**计算。

小规模子问题 → 中等规模子问题 → 原问题

在矩阵链乘中，就是先算： $m[i, i]$ ，再算： $m[i, i + 1]$ ，然后算： $m[i, i + 2]$ ，直到最后算出： $m[1, n]$ 。

具体步骤：

- 首先设置 $m[i, i] = 0, i = 1, \dots, n$
- 随后计算 $m[1, 2], m[2, 3], \dots, m[n - 1, n]$
- 然后是 $m[1, 3], m[2, 4], \dots, m[n - 2, n]$
- ... 直到我们能够计算 $m[1, n]$

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

	1	2	3	4	5	6	
0							1
							2
	0						3
			0				4
				0			5
					0		6

接下来通过具体算法的伪代码，来看一看这个题的时间复杂度。首先输入的 p 是一个数组，这个数组记录着所有矩阵的维数，两个二维矩阵，分别记录每一个子问题的最优成本和每一个字问题的最优分割点。

由于嵌套了三层 for 循环，所以时间复杂度是 $O(n^3)$ ，空间复杂度是 $O(n^2)$ 。

```
MatrixChainOrder(p):
    n = length(p) - 1

    for i = 1 to n:
        m[i, i] = 0

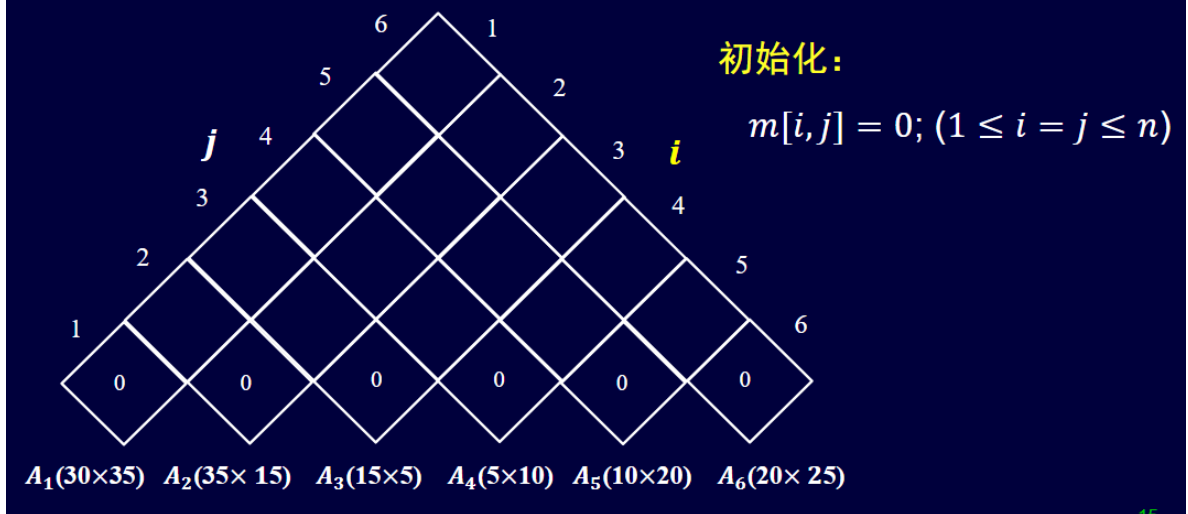
    for l = 2 to n:                // l 表示当前矩阵链长度
        for i = 1 to n - l + 1:    // i 表示区间起点
            j = i + l - 1          // j 表示区间终点
            m[i, j] = ∞

            for k = i to j - 1:     // 枚举最后一次分裂点
                q = m[i, k] + m[k+1, j] + p[i-1]p[k]p[j]

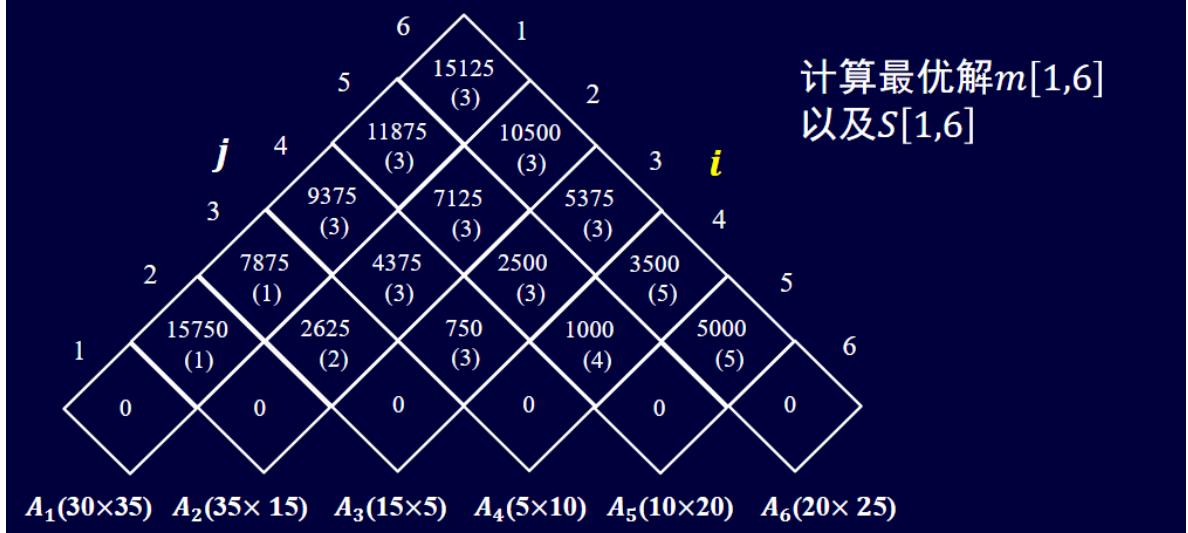
                if q < m[i, j]:
                    m[i, j] = q      // 记录最小成本
                    s[i, j] = k      // 记录最佳分裂点

    return m, s
```

■ Step4: 根据递归式，自底向上地计算最优解的值



■ Step4: 根据递归式，自底向上地计算最优解的值



Step 5: 根据记录的信息构造最优解

动态规划在计算过程中，通常还要**额外记录决策信息**。在矩阵链乘中： $m[i, j]$ 记录最小计算成本； $s[i, j]$ 记录最佳分裂点。最后根据 s 表还原最优括号化方案。

这一步我们已经有了所有信息的记录了，所以直接使用递归函数把结果 *print* 出来就好了。

```
PrintOptimalParens(s, i, j):
    if i == j:
        print Ai
    else:
        print "("
        PrintOptimalParens(s, i, s[i, j])
```

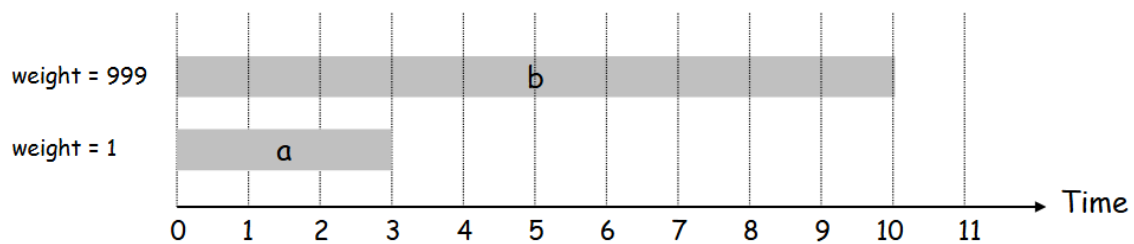
```
PrintOptimalParens(s, s[i,j] + 1, j)
print ")"
```

打印最优括号化方案的过程相当于**遍历一棵二叉树**，这棵二叉树有 n 个叶子节点，对应 n 个矩阵，所以总结点数是 $O(n)$ ，所以最终打印结果的时间复杂度也为 $O(n)$ 。

[06dynamic-programming.pdf](#)

带权的区间调度问题：

在贪心算法里，我们讲了不少带权的区间调度问题，那个时候我们给出的解法是结束时间优先，但这样的算法在任务**有权重**的时候不可以，可以看这个反例：



这个问题和高程中的 *BestCoverage* 问题其实本质上是一样的，对于每一个问题，我们有**选和不选两种情况**，所以这里也可以采用同样的动态规划思路。遵循上面的五步走方法来：

- *step1*: **定义子问题空间**

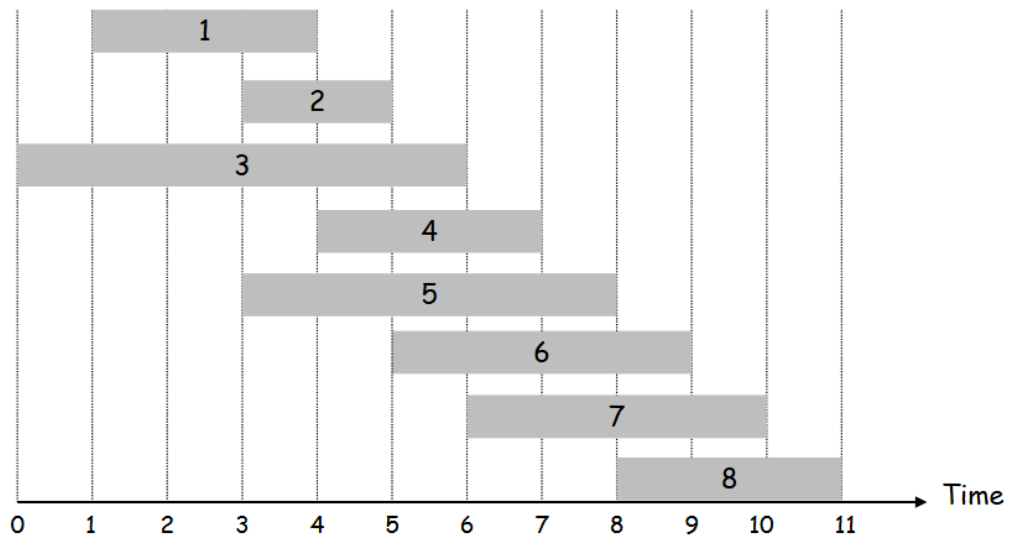
首先将所有任务按照结束时间从小到大排序：

$$f_1 \leq f_2 \leq \dots \leq f_n$$

我们定义： $OPT(j)$ 表示：只考虑前 j 个任务时，能够取得的最大总权重。也就是说：

$$OPT(j) = \text{任务 } 1, 2, \dots, j \text{ 中互不冲突任务集合的最大权重}$$

所以原问题的答案就是： $OPT(n)$



为了写递推式，需要定义一个辅助数组： $p(j)$ 表示：在任务 j 之前，和任务 j 兼容的、编号最大的任务。由于任务已经按照结束时间排序，所以：

$$p(j) = \max\{i < j \mid f_i \leq s_j\}$$

也就是说， $p(j)$ 是最后一个满足“结束时间不晚于任务 j 开始时间”的任务。如果任务 j 前面没有任何任务与它兼容，则：

$$p(j) = 0$$

其中 $OPT(0) = 0$ ，表示没有任务时最大权重为 0。

- **step2: 刻画最优解的特征结构**

对于最后一个任务 j ，最优解只有两种情况：

① **选择该任务 j** 。那么所有与任务 j 冲突的任务都不能选。那么前面还能继续考虑的任务只剩 $1, 2, \dots, p(j)$ ，因此这一种情况的总价值为：

$$v_j + OPT(p(j))$$

② **不选择该任务 j** 。只考虑前 $j - 1$ 个任务的最优解，因此这一种情况的总价值为：

$$OPT(j - 1)$$

- **step3: 根据最优子结构写递归式**

根据上面的二选一分析，可以得到递推式：

$$OPT(j) = \begin{cases} 0, & j = 0 \\ \max\{v_j + OPT(p(j)), OPT(j-1)\}, & j \geq 1 \end{cases}$$

递推式的核心就是： $OPT(j) = \max\{\text{选 } j, \text{不选 } j\}$

- step4: 自底向上计算最优值

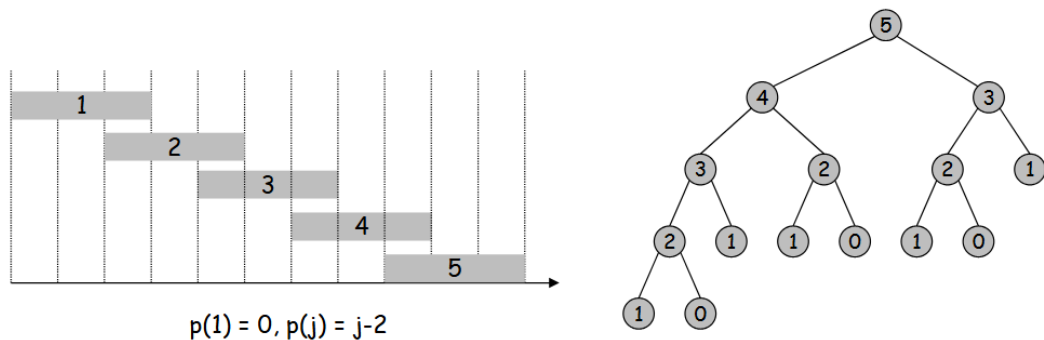
我们先看看如果直接使用传统的递归算法，会是一个什么样的结果：

```
Input:  $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$ 

Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .

Compute  $p(1), p(2), \dots, p(n)$ 

Compute-Opt( $j$ ) {
    if ( $j = 0$ )
        return 0
    else
        return  $\max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j-1))$ 
}
```



重复子问题 例如：

Compute-Opt(3) 被调用了 2 次；Compute-Opt(2) 被调用了 3 次

Compute-Opt(1) 被调用了 5 次；Compute-Opt(0) 被调用了 3 次

会有大量的重复计算出现，这些重复计算就是递归算法时间复杂度过高的本质原因。所以我们用排序 + 数组记录 + 动态规划来实现这道题。

```
Iterative-Compute-Opt:
    M[0] = 0
```

```

for j = 1 to n:
    M[j] = max(v[j] + M[p[j]], M[j-1])

return M[n]

```

排序消耗 $O(n \log n)$ ，计算 p 数组的复杂度为 $O(n \log n)$ ，主循环复杂度为 $O(n)$ ，所以总复杂度为：

$$O(n \log n)$$

- **step5: 根据记录信息记录最优解**

动态规划数组 M 只能告诉我们最优值是多少。如果还想知道具体选择了哪些任务，需要从 $j = n$ 开始反向追踪。对于任务 j ，我们比较： $v_j + M[p(j)]$ 和 $M[j - 1]$ 的值，如果选择 j 更优，那我们就输出任务 j ，and vice versa。

```

Find-Solution(j):
    if j == 0:
        return

    if v[j] + M[p[j]] > M[j-1]:
        print j
        Find-Solution(p[j])
    else:
        Find-Solution(j-1)

```

递归算法优化版：

但其实，递归问题也没我们想的那么差，我们上面也说了，递归消耗过大的本质是因为出现了大量的重复计算，如果我们能消除这一痛点，递归也可以变得简洁高效（本身就很简洁了）。

最原始的递推算法是：

```

Compute-Opt(j):
    if j == 0:
        return 0
    else:
        return max(v[j] + Compute-Opt(p[j]), Compute-Opt(j-1))

```


优化方法很简单：第一次算出 $OPT(j)$ 后，把它存到数组 $M[j]$ 里。以后再需要 $OPT(j)$ ，直接查表，不再递归计算。这样的优化就是 **Memoization，也就是记忆化搜索**。于是新的递归算法可以写为：

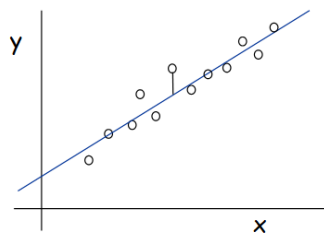
```
M-Compute-Opt(j):
    if M[j] is empty:
        M[j] = max(v[j] + M-Compute-Opt(p[j]), M-Compute-Opt(j-1))
    return M[j]      //记得设置M[0] = 0
```

每个 $OPT(j)$ 最多真正计算一次。之后再遇到它，都是 $O(1)$ 查表。所以递归主体的复杂度从指数级降低为 $O(n)$ 。

分段最小二乘问题（多路选择）

如果点为：

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$



用直线 $y = ax + b$ 拟合，那么误差平方和为：

$$SSE = \sum_{i=1}^n (y_i - ax_i - b)^2$$

$$a = \frac{n \sum_i x_i y_i - (\sum_i x_i)(\sum_i y_i)}{n \sum_i x_i^2 - (\sum_i x_i)^2}, \quad b = \frac{\sum_i y_i - a \sum_i x_i}{n}$$

我们的目标就是让这个值最小。但是现实中，很多数据不是整体接近一条直线，而是呈现出**分段线性趋势**。所以我们希望用若干条线段分别拟合不同区间内的点。

矛盾在于，我们既想要模型有效，也想让模型简洁，避免过拟合，因此用目标函数 $E + cL$ 来平衡二者。我们追求的最终目的是 $\min$ 目标函数

其中 E 表示所有分段内误差平方和的总和； L 表示使用的线段条数； $c > 0$ 表示每多使用一条线段要付出的固定惩罚。(有没有再次幻视拉格朗日乘子法)。接下来，依旧用动态规划五步走来做：

step1: 定义子问题空间

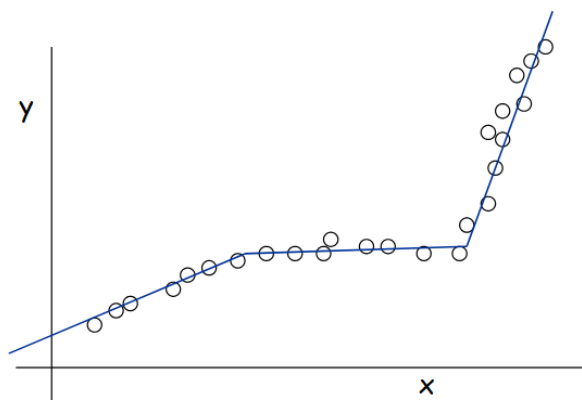
我们先定义 $OPT(j)$ 表示对前 j 个点 $p_1, p_2, \dots, p_j$ 进行最优分段拟合时的最小总成本。于是，原问题的答案就是 $OPT(n)$ 。

同时，定义 $e(i, j)$ 表示点 $p_i, p_{i+1}, \dots, p_j$ 被一条直线拟合时的最小误差平方和。这个量可以提前计算出来，作为动态规划中的辅助信息。

step2: 刻画最优解的结构特征

考虑 $OPT(j)$ ，也就是前 j 个点的最优分段方案。我们重点看这个最优方案的最后一段。

假设最后一段从第 i 个点开始，最后一段为 $p_i, p_{i+1}, \dots, p_j$ 。那么前面的点 $p_1, p_2, \dots, p_{i-1}$ 必须采用最优分段方案。



如果前 $i - 1$ 个点的分段方案不是最优的，那么就可以把这一部分替换成更优方案，而最后一段 $p_i, \dots, p_j$ 保持不变。这样总成本会降低，和原方案已经是 $p_1, \dots, p_j$ 的最优方案矛盾。

因此，该问题具有最优子结构：如果最后一段确定为 $p_i, \dots, p_j$ ，那么前面的 $p_1, \dots, p_{i-1}$ 必须也是最优分段方案。大问题的最优是由小问题的最优拼起来的。

step3: 根据最优子结构写递推式

整个方案的成本由三部分组成：前 $i - 1$ 个点的最优成本 $OPT(i - 1)$ ，最后一段的拟合误差 $e(i, j)$ ，以及新增一条线段的惩罚 c 。

$$OPT(j) = \begin{cases} 0, & j = 0, \\ \min_{1 \leq i \leq j} \{OPT(i-1) + e(i, j) + c\}, & j \geq 1. \end{cases}$$

递推式的含义是：为了求前 j 个点的最优分段方案，我们枚举最后一段的起点 i ，把问题拆成“前 $i-1$ 个点的最优分段”加上“最后一段 $p_i, \dots, p_j$ ”的成本，然后选择总代价最小的方案。

step4: 自底向上计算最优值

令 $M[j] = OPT(j)$ 。初始化 $M[0] = 0$ ，表示没有点时成本为 0。然后按照 $j = 1, 2, \dots, n$ 的顺序依次计算 $M[j]$ 。

Segmented-Least-Squares:

```
M[0] = 0

for j = 1 to n:
    M[j] = +∞
    for i = 1 to j:
        cost = M[i-1] + e(i, j) + c
        if cost < M[j]:
            M[j] = cost
            S[j] = i

return M[n]
```

两层循环中，所有可能的区间 $[i, j]$ 一共有 $O(n^2)$ 个。如果计算 $e(i, j)$ ，每次需要遍历区间内的点，最坏需要 $O(n)$ 时间，因此总的时间复杂度为 $O(n^3)$ 。

$$a_{ij} = \frac{N \cdot \sum_{k=i}^j x_k y_k - \left(\sum_{k=i}^j x_k\right) \left(\sum_{k=i}^j y_k\right)}{N \cdot \sum_{k=i}^j x_k^2 - \left(\sum_{k=i}^j x_k\right)^2} \quad b_{ij} = \frac{\sum_{k=i}^j y_k - a_{ij} \cdot \sum_{k=i}^j x_k}{N}$$

$$N = j - i + 1$$

$$e_{ij} = \sum_{k=i}^j (y_k - (a_{ij} x_k + b_{ij}))^2$$

step5: 根据记录信息恢复最优解

如果只需要最优值，返回 $M[n]$ 即可。但如果要知道具体如何分段，就需要利用数组 S 从后往前恢复。

假设 $S[j] = i$ ，说明前 j 个点的最优方案中，最后一段是 $p_i, \dots, p_j$ 。然后继续恢复前 $i-1$ 个点的最优方案，即转到 $S[i-1]$ 。不断重复，直到 $j = 0$ 。

```
Find-Segments(j):
    if j == 0:
        return

    i = S[j]
    Find-Segments(i-1)
    output segment (i, j)
```

带权区间调度问题中，每个任务只有“选”或“不选”两种情况，属于二选一决策。

而在分段最小二乘问题中，计算 $OPT(j)$ 时，需要决定“最后一段从哪里开始”。最后一段可能是 p_j ，也可能是 p_{j-1}, p_j ，一直到 $p_1, \dots, p_j$ 。

也就是说，最后一段起点 i 可以取 $1, 2, \dots, j$ 中的任意一个。因此该问题属于 *multiway choice*，即多路选择型动态规划。

前缀计算的优化

上面的复杂度分析中，我们发现光是计算误差项就要 $O(n)$ 的时间复杂度，造成了很大的消耗，所以从这个痛点入手，我们可以对效率进行优化：

可以预处理若干前缀和数组，例如 $S_x[t] = \sum_{k=1}^t x_k$, $S_y[t] = \sum_{k=1}^t y_k$, $S_{xx}[t] = \sum_{k=1}^t x_k^2$, $S_{xy}[t] = \sum_{k=1}^t x_k y_k$ 。

有了这些前缀和之后，任意区间 $[i, j]$ 内的统计量都可以在 $O(1)$ 时间得到。例如：

$$\sum_{k=i}^j x_k = S_x[j] - S_x[i-1], \quad \sum_{k=i}^j y_k = S_y[j] - S_y[i-1].$$

类似地， $\sum_{k=i}^j x_k^2$ 和 $\sum_{k=i}^j x_k y_k$ 也可以通过前缀和在 $O(1)$ 时间得到。

如果所有 $e(i, j)$ 都能在 $O(1)$ 时间内得到，则预处理误差项的总时间可降为 $O(n^2)$ ，动态规划部分也是 $O(n^2)$ ，因此整体时间复杂度可以优化到 $O(n^2)$ 。

非连续背包问题 Knapsack Problem

给定 n 个物品和一个容量为 W 的背包。第 i 个物品有重量 $w_i > 0$ ，价值 $v_i > 0$ 。每个物品只能选择一次，要么放入背包，要么不放入背包。目标是在总重量不超过 W 的前提下，使总价值最大。

这类问题也被叫作 **0/1 背包问题**。这里的“0/1”表示每个物品只有两种状态：**选，或者不选**。

Ex: { 3, 4 } has value 40.

W = 11

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

Greedy: repeatedly add item with maximum ratio v_i / w_i .

Ex: { 5, 2, 1 } achieves only value = 35 $\Rightarrow$ greedy not optimal.

很容易发现，由于此时物品不可以在被切割，贪心算法找不到最优解了，因为有可能出现比较极端的情况，比如说某一个单独的物品太大之类的.....所以背包问题不能只看局部性价比，因为当前看似最划算的物品，可能占掉容量，导致后面无法组合出更优方案。

依旧用动态规划五步走的方法来做：

step1: 定义子问题空间

从直觉上，我们可能先尝试定义 $OPT(i)$ 表示：只考虑前 i 个物品时，能够得到的最大价值。但问题也很明显：**我们不知道背包还剩多少容量**。

假设要选物品 i ，它能不能放进去，不只取决于前 $i - 1$ 个物品的最大价值，还取决于前面已经用了多少重量。仅仅用 $OPT(i)$ 记录“前 i 个物品的最大价值”，没有记录容量信息，所以无法判断物品 i 是否能被加入。

所以我们需要给状态增加一个变量→**把背包的容量也加入子问题！**

定义 $OPT(i, w)$ 表示：只考虑前 i 个物品，并且背包容量限制为 w 时，能够取得的最大价值。这里的 w 不是某个物品的重量，而是当前**子问题中的容量上限**。**原问题的答案就是 $OPT(n, W)$**

step2: 刻画最优解的结构特征

对 $OPT(i, w)$ 的最优解，若不选物品 i ，则它必须是 $OPT(i - 1, w)$ 的最优解；若选物品 i ，则去掉物品 i 后的剩余部分必须是 $OPT(i - 1, w - w_i)$ 的最优解。否则都可以用更优子解替换，从而得到更优的大问题解，产生矛盾。

step3: 根据最优子结构写递推式

根据上面的分析，可以得到背包问题的递推式：

$$OPT(i, w) = \begin{cases} 0, & i = 0, \\ OPT(i - 1, w), & w_i > w, \\ \max\{OPT(i - 1, w), v_i + OPT(i - 1, w - w_i)\}, & w_i \leq w. \end{cases}$$

没有物品可选时，最大价值为 0；如果第 i 个物品太重，放不进容量为 w 的背包，就只能不选它；如果第 i 个物品放得进去，就比较“选它”和“不选它”两种方案，取价值较大者。

step4: 自底向上计算最优值

令 $M[i, w] = OPT(i, w)$ 。因为状态有两个变量，所以要填一个二维表。行表示考虑前多少个物品，列表示当前容量限制。

```
Knapsack:
  for w = 0 to W:
    M[0, w] = 0

  for i = 1 to n:
    for w = 0 to W:
      if w_i > w:
        M[i, w] = M[i-1, w]
      else:
        M[i, w] = max(M[i-1, w], v_i + M[i-1, w-w_i])

  return M[n, W]
```

		W + 1											
		0	1	2	3	4	5	6	7	8	9	10	11
i + 1	ϕ	0	0	0	0	0	0	0	0	0	0	0	0
	{1}	0	1	1	1	1	1	1	1	1	1	1	1
	{1, 2}	0	1	6	7	7	7	7	7	7	7	7	7
	{1, 2, 3}	0	1	6	7	7	18	19	24	25	25	25	25
	{1, 2, 3, 4}	0	1	6	7	7	18	22	24	28	29	29	40
	{1, 2, 3, 4, 5}	0	1	6	7	7	18	22	28	29	34	34	40

OPT: { 4, 3 }
value = 22 + 18 = 40

W = 11

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

step5: 根据记录信息恢复最优解

设当前在 $M[i, w]$ 。如果 $M[i, w] = M[i - 1, w]$, 说明最优解**可以不选**物品 i , 于是转到 $M[i - 1, w]$ 。如果 $M[i, w] \neq M[i - 1, w]$, 说明最优解**选择**了物品 i , 于是记录物品 i , 并转到 $M[i - 1, w - w_i]$ 。

```
Find-Solution(i, w):  
    if i == 0:  
        return  
  
    if M[i, w] == M[i-1, w]:  
        Find-Solution(i-1, w)  
    else:  
        print i  
        Find-Solution(i-1, w-w_i)
```

时间复杂度分析（这个其实是伪多项式哦！）

二维表 M 的规模是 $(n + 1) \times (W + 1)$, 每个格子只需要 $O(1)$ 时间计算, 所以时间复杂度是 $\Theta(nW)$ 。

但**容量 W 可以用 $\log W$ 位**写出来, 因此 $\Theta(nW)$ 不是严格意义上的多项式时间, 而是伪多项式时间。

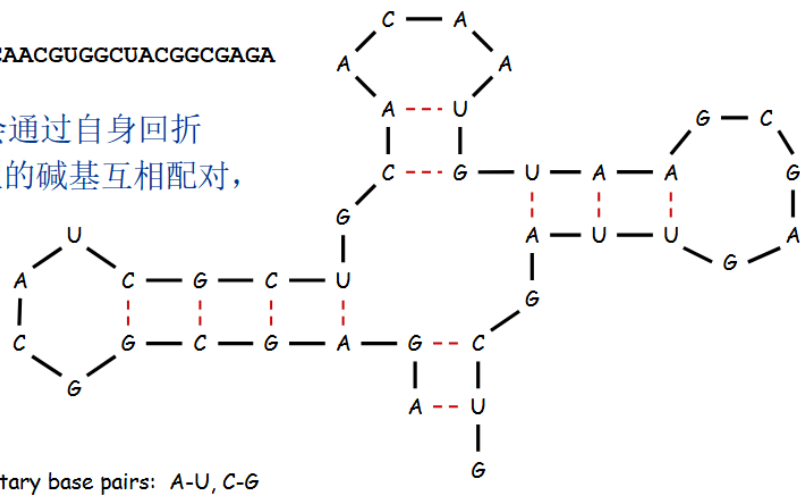
因为在严格的算法理论里, 多项式时间通常指: 关于输入长度的多项式。而 W 的输入长度大约是 $\log W$, 不是 W 。比如 $W = 1,000,000$, 用二进制表示只需要大约 20 位, 但要开 1000000 列的表来计算, 容量 W 的数值一旦很大, 背包 DP 的表就会爆炸。

也许我们会疑问, 前面那么多出现了 n 规模的输入, 都是正常的直接写 $O(n)$ 、 $O(n \log n)$ 之类的复杂度, 为什么从来没有特殊地讨论“**位数**”的影响呢? 因为从本质上看, 这些 n 传入的是事件的**规模**, 比如循环 n 轮, 代表一样的事件重复做 n 次, 本质上是“**次数**”; 而输入的 W , 在问题中本质上是一个数值, 认为是“**参数**”。

RNA二级结构

Ex: GUCGAUUGAGCGAAUGUAACAACGUGGCUACGGCGAGA

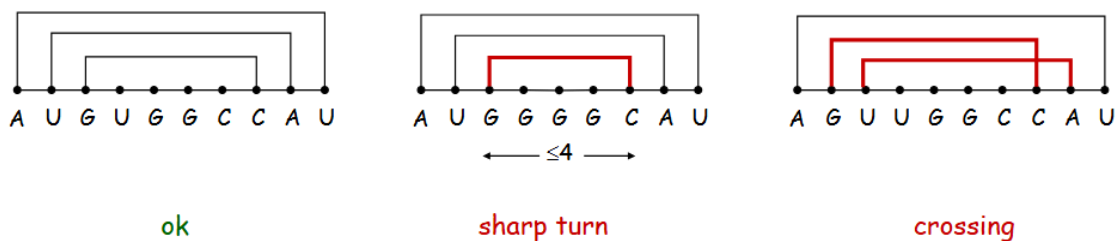
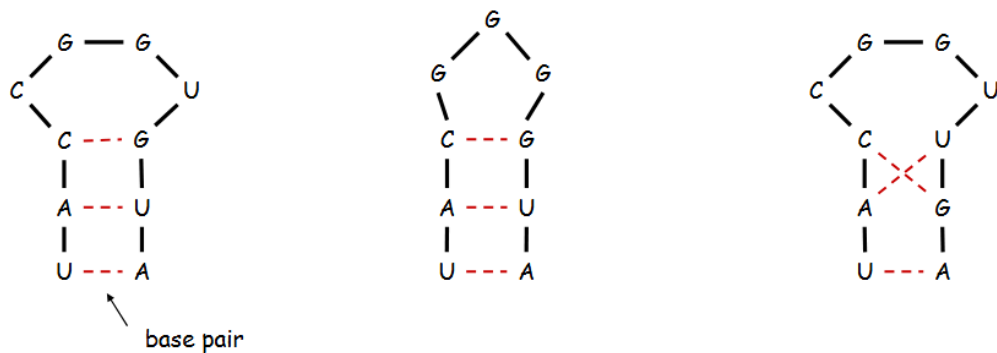
RNA 是单链分子，它会通过自身回折（loop back），让链上的碱基互相配对，形成稳定的局部碱基对结构，这些配对关系和空间排布就构成了二级结构。



这个问题适合动态规划，关键原因是 **non-crossing 条件**。假设最后一个碱基 b_j 和某个前面的碱基 b_t 配对。由于 **配对不能交叉**，那么整个问题会自然分成两个互不干扰的区间：

- 左边区间: $b_i, \dots, b_{t-1}$;
- 中间区间: $b_{t+1}, \dots, b_{j-1}$ 。

这两个区间内部可以分别继续寻找最优配对，而且它们之间不会因为交叉配对互相干扰。



step1: 定义子问题空间

我们需要定义区间状态 $OPT(i, j)$ 表示子串 $b_i, b_{i+1}, \dots, b_j$ 中最多可以形成多少个合法碱基对。原问题的答案就是 $OPT(1, n)$ 。

step2: 刻画最优解的结构特征

为了分析最优结构，我们观察区间右端点 b_j 。在最优结构中， b_j 只有两种情况：**不参与配对**，或者与前面的某个碱基 b_t 配对。

- 如果 b_j 不参与配对：

那么所有配对都发生在子串 $b_i, \dots, b_{j-1}$ 内。因此，此时大问题 $OPT(i, j)$ 的最优解应该由子问题 $OPT(i, j-1)$ 的最优解构成。

- 如果 b_j 与 b_t 配对，则必须满足 $i \leq t < j-4$ ，并且 b_t 与 b_j 是互补碱基对。

此时，由于 *non-crossing* 条件，任何配对都不能跨过 (b_t, b_j) 。所以区间 $b_i, \dots, b_j$ 会被拆成两个互不干扰的子区间：**左侧**区间 $b_i, \dots, b_{t-1}$ ，以及**内部**区间 $b_{t+1}, \dots, b_{j-1}$ 。如果最优解中包含配对 (b_t, b_j) ，那么剩余部分必须分别是这两个子区间的最优结构，即来自 $OPT(i, t-1)$ 和 $OPT(t+1, j-1)$ 。

step3: 根据最优子结构写递推式

如果区间太短， $i \geq j-4$ ，区间内无法形成合法配对，因此 $OPT(i, j) = 0$ 。

如果区间足够长，则需要比较两类方案：**第一类是 b_j 不参与配对**，此时配对数为 $OPT(i, j-1)$ ；**第二类是 b_j 与某个 b_t 配对**，此时配对数为 $1 + OPT(i, t-1) + OPT(t+1, j-1)$ 。因此递推式为：

$$OPT(i, j) = \begin{cases} 0, & i \geq j-4, \\ \max \left\{ OPT(i, j-1), \max_{\substack{i \leq t < j-4 \\ b_t \text{ 与 } b_j \text{ 互补}}} (1 + OPT(i, t-1) + OPT(t+1, j-1)) \right\}, & i < j-4. \end{cases}$$

step4: 自底向上计算最优值

令 $M[i, j] = OPT(i, j)$ 。由于 $OPT(i, j)$ 依赖于更短区间，例如 $OPT(i, j-1)$ 、 $OPT(i, t-1)$ 和 $OPT(t+1, j-1)$ ，所以需要按照区间长度从小到大计算。

首先，对于所有满足 $i \geq j-4$ 的短区间，初始化 $M[i, j] = 0$ 。然后按照区间长度递增的顺序填表。

```
RNA-Secondary-Structure:
  for all i, j with i >= j - 4:
    M[i, j] = 0

  for d = 5 to n - 1:
    for i = 1 to n - d:
```

```

    j = i + d
    M[i,j] = M[i,j-1]

    for t = i to j - 5:
        if b[t] and b[j] are complementary:
            q = 1 + M[i,t-1] + M[t+1,j-1]
            if q > M[i,j]:
                M[i,j] = q
                S[i,j] = t

    return M[1,n]

```

其中, $d = j - i$ 表示区间长度差, i 表示区间起点, $j = i + d$ 表示区间终点, t 用来枚举可能与 b_j 配对的位置。

step5: 根据记录信息恢复最优解

可以用 $S[i, j]$ 记录最优方案中 b_j 的选择。如果 $S[i, j]$ 为空, 表示 b_j 不参与配对, 此时继续追踪区间 $[i, j - 1]$ 。如果 $S[i, j] = t$, 表示最优方案中 b_t 与 b_j 配对, 此时输出配对 (t, j) , 并继续追踪两个子区间 $[i, t - 1]$ 和 $[t + 1, j - 1]$ 。

```

Find-Structure(i, j):
    if i >= j - 4:
        return

    if S[i,j] is empty:
        Find-Structure(i, j-1)
    else:
        t = S[i,j]
        output (t, j)
        Find-Structure(i, t-1)
        Find-Structure(t+1, j-1)

```

复杂度分析:

状态 $OPT(i, j)$ 对应所有区间 $[i, j]$, 共有 $O(n^2)$ 个状态。计算每个状态时, 需要枚举可能与 b_j 配对的位置 t , 最多有 $O(n)$ 种选择。因此, 总时间复杂度为 $O(n^3)$ 。

由于需要保存二维表 $M[i, j]$, 空间复杂度为 $O(n^2)$ 。

编辑距离 Edit Distance

How similar are two strings?

- **ocurrence** (拼写有误)
- **occurrence** (正确拼写)

o	c	u	r	r	a	n	c	e	-
o	c	c	u	r	r	e	n	c	e

6 mismatches, 1 gap

如果两个字符不同, 就产生 *mismatch*, 即“不匹配代价”; 如果某个字符和空位对齐, 就产生 *gap*, 即“空位代价”。

所以编辑距离衡量的是: 两个字符串通过插入、删除、替换等操作变得一致所需要付出的**最小代价**。

- **空位罚分 (Gap penalty)** δ : 插入 / 删除一个字符的代价。
 - **错配罚分 (Mismatch penalty)** α_{pq} : 字符 p 和 q 不匹配时的代价
- 总代价 = 所有空位罚分 + 所有错配罚分之和。

C	T	G	A	C	C	T	A	C	C	T
C	C	T	G	A	C	T	A	C	A	T

$\alpha_{TC} + \alpha_{GT} + \alpha_{AG} + 2\alpha_{CA}$

-	C	T	G	A	C	C	T	A	C	C	T
C	C	T	G	A	C	-	T	A	C	A	T

$2\delta + \alpha_{CA}$

一个合法对齐可以理解为: 从字符串 X 中选一些字符, 与字符串 Y 中的一些字符配对。但配对不能乱来, 必须保持原来的顺序。

step1: 定义子问题空间

定义 $OPT(i, j)$ 表示字符串 X 的前 i 个字符 $x_1, \dots, x_i$ 和字符串 Y 的前 j 个字符 $y_1, \dots, y_j$ 进行最优对齐时的最小代价。原问题的答案就是 $OPT(m, n)$ 。

step2: 刻画最优解的结构特征

我们重点看两个字串的最后字符 x_i 和 y_j , 对于最优对齐方案中, 最后一步只有三种可能。

- **情况一: x_i 和 y_j 配对**

如果 x_i 和 y_j 配对, 那么这一步产生的代价是 $\alpha_{x_i y_j}$ 。剩下的问题就是对齐 $x_1, \dots, x_{i-1}$ 和 $y_1, \dots, y_{j-1}$, 其最优代价为 $OPT(i-1, j-1)$ 。因此, 这种情况的总代价是 $\alpha_{x_i y_j} + OPT(i-1, j-1)$ 。

- **情况二: x_i 不和任何字符配对**

如果 x_i 不和 Y 中的字符配对, 那么它只能和空位 - 对齐, 产生一个 gap 代价 δ 。剩下的问题是对齐 $x_1, \dots, x_{i-1}$ 和 $y_1, \dots, y_j$, 代价为 $OPT(i-1, j)$ 。因此, 这种情况的总代价是 $\delta + OPT(i-1, j)$ 。

• **情况三: y_j 不和任何字符配对**

同理, 如果 y_j 不和 X 中的字符配对, 那么 y_j 和空位对齐, 产生 gap 代价 δ 。剩下的问题是对齐 $x_1, \dots, x_i$ 和 $y_1, \dots, y_{j-1}$, 代价为 $OPT(i, j-1)$ 。因此, 这种情况的总代价是 $\delta + OPT(i, j-1)$ 。

step3: 根据最优子结构写递推式

$$OPT(i, j) = \begin{cases} j\delta & \text{if } i=0 \\ \min \begin{cases} \alpha_{x_i y_j} + OPT(i-1, j-1) \\ \delta + OPT(i-1, j) \\ \delta + OPT(i, j-1) \end{cases} & \text{otherwise} \\ i\delta & \text{if } j=0 \end{cases}$$

$i=0$ 或 $j=0$: 全部空位罚分

其它情况:

- x_i 和 y_j 配对
- y_j 后插入空位与 x_i 配对
- x_i 后插入空位与 y_j 配对

step4: 自底向上计算最优值

令 $M[i, j] = OPT(i, j)$ 。由于 $M[i, j]$ 依赖左上方 $M[i-1, j-1]$ 、上方 $M[i-1, j]$ 和左方 $M[i, j-1]$, 所以可以从左上角开始逐行或逐列填表。

```
Sequence-Alignment:
for i = 0 to m:
    M[i, 0] = i * δ

for j = 0 to n:
    M[0, j] = j * δ

for i = 1 to m:
    for j = 1 to n:
        M[i, j] = min(
            α[x_i, y_j] + M[i-1, j-1],
            δ + M[i-1, j],
            δ + M[i, j-1]
        )

return M[m, n]
```

最终 $M[m, n]$ 就是两个字符串之间的最小编辑距离或最小对齐代价。

step5: 根据记录信息恢复最优解

如果只需要最优代价, 返回 $M[m, n]$ 即可。如果想知道具体怎么对齐, 就需要从 $M[m, n]$ 反向追踪。

从位置 (i, j) 开始:

- 如果 $M[i, j] = \alpha_{x_i y_j} + M[i - 1, j - 1]$, 说明 x_i 和 y_j 配对, 然后转到 $(i - 1, j - 1)$ 。
- 如果 $M[i, j] = \delta + M[i - 1, j]$, 说明 x_i 和空位配对, 然后转到 $(i - 1, j)$ 。
- 如果 $M[i, j] = \delta + M[i, j - 1]$, 说明 y_j 和空位配对, 然后转到 $(i, j - 1)$ 。
- 一直追踪到 $i = 0$ 或 $j = 0$, 就可以恢复出一个最优对齐方案。

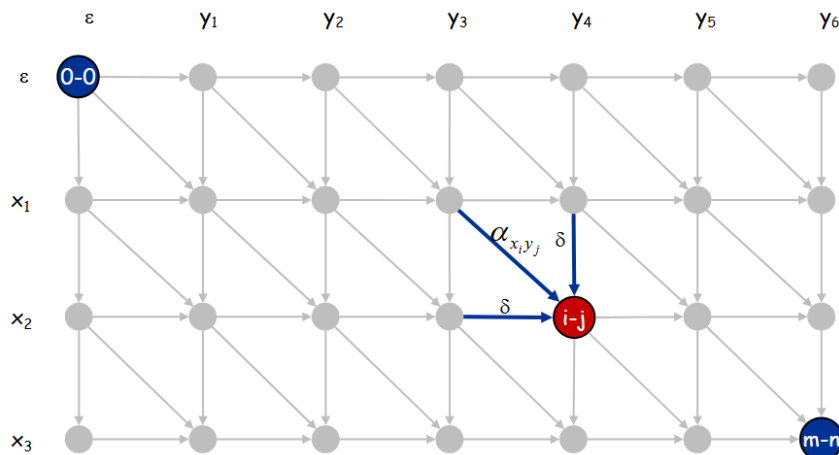
复杂度分析:

DP 表共有 $(m + 1)(n + 1)$ 个状态。每个状态只需要比较三种情况, 计算时间为 $O(1)$ 。因此, 时间复杂度为 $\Theta(mn)$, 空间复杂度也是 $\Theta(mn)$ 。

时间复杂度经证明, 已经近乎达到了最优, 但是空间复杂度我们并不满意, 下面是一种**有效降低空间复杂度**的解法。

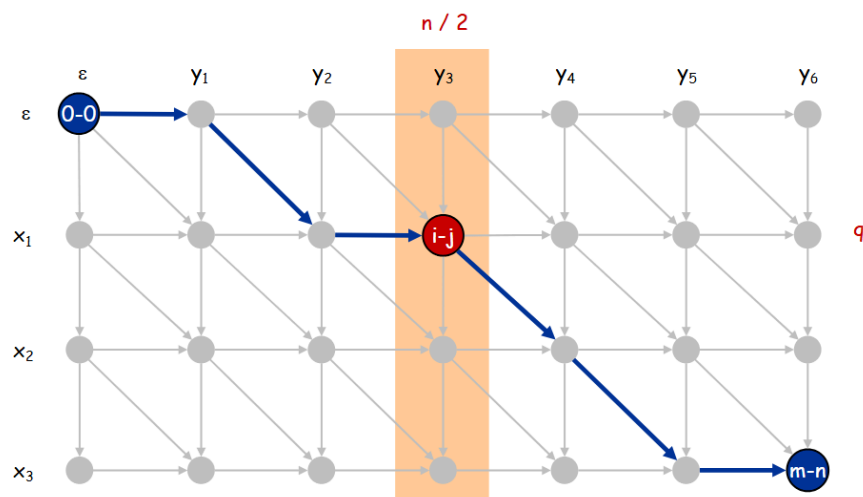
编辑距离→另一种很天才的解法:

把思路拉到了图世界中, 并总体采用了**最短路径 + 分治的思路**:

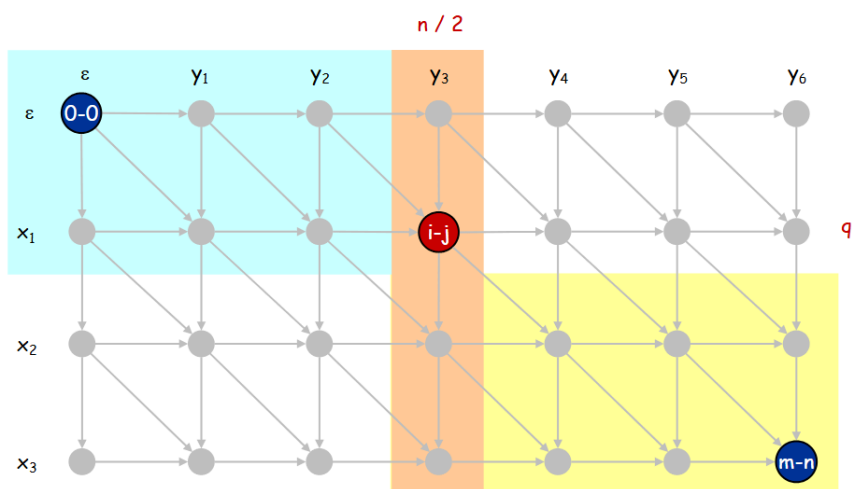


移动方向	含义	代价
对角线: $(i-1, j-1) \rightarrow (i, j)$	x_i 与 y_j 配对 (匹配或不匹配)	不匹配代价 $\alpha x_i y_j$ (匹配时代价为 0)
向下: $(i-1, j) \rightarrow (i, j)$	x_i 未配对 (y_j 后插入一个空位)	空位代价 δ
向右: $(i, j-1) \rightarrow (i, j)$	y_j 未配对 (x_i 后插入一个空位)	空位代价 δ

编辑距离问题可以看成从 $(0, 0)$ 到 (m, n) 的最短路径问题。为了在不保存整张 DP 表的情况下恢复最优路径, 可以采用分治思想。首先取中间列 $j = n/2$ 。最优路径必然经过该列的某个点 $(q, n/2)$, 因此需要找出这个点。



为此, **正向计算** $f(i, n/2)$, 表示从起点 $(0, 0)$ 到中间列点 $(i, n/2)$ 的最短代价; **反向计算** $g(i, n/2)$, 表示从中间列点 $(i, n/2)$ 到终点 (m, n) 的最短代价。对于中间列上的任意点 $(i, n/2)$, 经过该点的最短路径代价为 $f(i, n/2) + g(i, n/2)$ 。因此, **选择使该值最小的 q** , 即可确定最优路径经过的中间点 $(q, n/2)$ 。



找到中间点后, 将原问题分解为两个子问题: 从 $(0, 0)$ 到 $(q, n/2)$, 以及从 $(q, n/2)$ 到 (m, n) 。递归处理两个子问题, 即可恢复完整的最优对齐路径。该方法的核心是“**动态规划找中间**

切点，分治恢复最优路径”，时间复杂度仍为 $O(mn)$ ，但空间复杂度可以降为 $O(m + n)$ 。

$$T(m, n) \leq 2T(m, n/2) + O(mn) \Rightarrow T(m, n) = O(mn \log n)$$